

Использование языка программирования Julia для решения математических задач

Часть II

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Алексеев, Е. Р. Scilab: Решение инженерных и математических задач: учеб. издание. / Е.Р. Алексеев, К. В. Дога, О. В. Чеснокова М. – Базальт СПО; ДМК Пресс, 2024. – 440 с.
2. Балакин, А. А. Численные методы и математическое моделирование: Учебное пособие / А.А. Балакин. – Долгопрудный: Издательский Дом «Интеллект», 2022. – 288 с.
3. Деза, Е. И. Введение в криптографию: Теоретико-числовые основы защиты информации. Учебное пособие / Е. И. Деза, Л. В. Котова. – Москва: ЛЕНАНД, 2022. – 376 с.
4. Кнауб, Л. В. Теоретико-численные методы в криптографии / Л. В. Кнауб, Е. А. Новиков, Ю. А. Шитов; Сибирский федеральный университет – Красноярск: 2011. – 160 с.

Глава 19 Численное решение обыкновенных дифференциальных уравнений

При решении задач математического моделирования очень часто приходится решать обыкновенные дифференциальные уравнения.

Напомним численные методы решения дифференциального уравнения первого порядка. Будем рассматривать для следующей задачи Коши. Найти решение дифференциального уравнения

$$x' = f(x, t),$$

удовлетворяющего начальному условию

$$x(t_0) = x_0$$

иными словами, требуется найти интегральную кривую $x = x(t)$, проходящую через заданную точку $M_0(t_0, x_0)$.

Для дифференциального уравнения n -го порядка

$$x^{(n)} = f(t, x, x', x'', \dots, x^{(n-1)})$$

задача Коши состоит в нахождении решения $x = x(t)$, удовлетворяющего уравнению и начальным условиям

$$x(t_0) = x_0, x'(t_0) = x'_0, \dots, x^{(n-1)}(t_0) = x_0^{(n-1)}$$

Рассмотрим основные численные методы решения задачи Коши.

19.1 Решение дифференциальных уравнений методом Эйлера

При решении задачи Коши на интервале $[t_0, t_n]$, выбрав достаточно малый шаг h , построим систему точек

$$t_i = t_0 + ih, \quad i = 0, 1, \dots, n, \quad h = \frac{t_n - t_0}{n}$$

Основная расчётная формула

$$x_{i+1} = x_i + hf(x_i, t_i), \quad i = 0, 1, \dots, n - 1.$$

19.2 Решение дифференциальных уравнений при помощи модифицированного метода Эйлера

Более точным методом решения задачи Коши является *модифицированный метод Эйлера*, при котором сначала вычисляют промежуточные значения.

$$t_p = t_i + \frac{h}{2}, \quad x_p = x_i + \frac{h}{2}f(x_i, t_i),$$

после чего находят значение x_{i+1} по формуле

$$x_{i+1} = x_i + hf(x_p, t_p), \quad i = 0, 1, \dots, n - 1$$

19.3 Решение дифференциальных уравнений методами Рунге-Кутта

Рассмотренные выше методы Эйлера (как обычный, так и модифицированный) являются частными случаями явного *метода Рунге-Кутта* k -го порядка. В общем случае формула вычисления очередного приближения методом Рунге-Кутта имеет вид:

$$x_{i+1} = x_i + h\varphi(t_i, x_i, h), \quad i = 0, 1, \dots, n - 1$$

Метод Эйлера является методом Рунге-Кутта первого порядка ($k = 1$) и получается при $\varphi(t, x, h) = f(t, x)$.

Семейство *методов Рунге-Кутта* второго порядка имеет вид

$$x_{i+1} = x_i + h \left((1 - \alpha) f(t_i, x_i) + \alpha f \left(t_i + \frac{h}{2\alpha}, x_i + \frac{h}{2\alpha} f(t_i, x_i) \right) \right),$$

$$i = 0, 1, \dots, n - 1$$

Два наиболее известных среди методов Рунге-Кутты второго порядка – это метод Хойна ($\alpha = \frac{1}{2}$) и модифицированный метод Эйлера ($\alpha = 1$). При $\alpha = \frac{1}{2}$ получаем расчётную формулу *метода Хойна*:

$$x_{i+1} = x_i + \frac{h}{2} (f(t_i, x_i) + f(t_i + h, x_i + hf(t_i, x_i))), \quad i = 0, 1, \dots, n - 1$$

При $\alpha = 1$ получаем расчётную формулу уже рассмотренного выше модифицированного метода Эйлера

$$x_{i+1} = x_i + hf \left(t_i + \frac{h}{2}, x_i + \frac{h}{2} f(t_i, x_i) \right), \quad i = 0, 1, \dots, n - 1$$

Наиболее известным является *метод Рунге-Кутты четвёртого порядка*, расчётные формулы которого можно записать в виде:

$$\left\{ \begin{array}{l} x_{i+1} = x_i + \Delta x_i, \quad i = 0, 1, \dots, n - 1 \\ \Delta x_i = \frac{h}{6} (K_1^i + 2K_2^i + 2K_3^i + K_4^i) \\ K_1^i = f(t_i, x_i) \\ K_2^i = f(t_i + \frac{h}{2}, x_i + \frac{h}{2} K_1^i) \\ K_3^i = f(t_i + \frac{h}{2}, x_i + \frac{h}{2} K_2^i) \\ K_4^i = f(t_i + h, x_i + hK_3^i) \end{array} \right.$$

Рассмотренные методы Рунге-Кутты относятся к классу *одношаговых методов*, в которых для вычисления значения в очередной точке x_{k+1} нужно знать значение в предыдущей точке x_k .

Ещё один класс методов решения задачи Коши – *многошаговые методы*, в которых используются точки $x_{k-3}, x_{k-2}, x_{k-1}, x_k$ для вычисления x_{k+1} . В многошаговых методах первые четыре начальные точки $(t_0, x_0), (t_1, x_1), (t_2, x_2), (t_3, x_3)$ должны быть получены заранее любым из одношаговых методов (метод **Эйлера**, **Рунге-Кутты** и т.д.). Наиболее известными многошаговыми методами являются методы прогноза-коррекции **Адамса** и **Милна**.

19.4 Решение дифференциальных уравнений методом Адамса

Рассмотрим решение задачи Коши методом **Адамса**, будем численно решать дифференциальное на интервале $[t_i, t_{i+1}]$, считая, что решение в точках $t_0, t_1, t_2, \dots, t_i$ уже найдено (метод Эйлера или Рунге-Кутта), и значения в этих точках будем использовать для нахождения значения $x(t_{i+1})$.

Первое приближение (прогноз) $\tilde{x}_{i+1}$ вычисляется по формуле.

$$\tilde{x}_{i+1} = x_i + \frac{h}{24}(-9f(t_{i-3}, x_{i-3}) + 37f(t_{i-2}, x_{i-2}) - 59f(t_{i-1}, x_{i-1}) + 55f(t_i, x_i))$$

Как только $\tilde{x}_{i+1}$ вычислено, его можно использовать для вычисления второго приближения (корректор)

$$x_{i+1} = x_i + \frac{h}{24}(f(t_{i-2}, x_{i-2}) - 5f(t_{i-1}, x_{i-1}) + 19f(t_i, x_i) + 9f(t_{i+1}, \tilde{x}_{i+1}))$$

Таким образом, для вычисления значения $x(t_{i+1})$ методом **Адамса** необходимо последовательно применять формулы для вычисления прогноза $\tilde{x}_{i+1}$ и корректора x_{i+1} , а первые четыре точки можно получить односточечными методами **Эйлера** или **Рунге-Кутта**.

19.5 Решение дифференциальных уравнений методом Милна

Рассмотрим решение задачи Коши методом **Милна**, будем численно решать дифференциальное на интервале $[t_k, t_{k+1}]$, считая, что решение в точках $t_0, t_1, t_2, \dots, t_k$ уже найдено (метод Эйлера или Рунге-Кутта), и значения в этих точках будем использовать для нахождения значения $x(t_{k+1})$. Вычислительная схема метода **Милна** состоит в следующем.

Вычисляем первое приближение - **прогноз Милна** $\tilde{x}_{k+1}$ для значения функции в точке t_{k+1}

$$\tilde{x}_{k+1} = x_{k-3} + \frac{4h}{3}(2f(t_{k-2}, x_{k-2}) - f(t_{k-1}, x_{k-1}) + 2f(t_k, x_k))$$

Далее вычисляем второе приближение - **корректор Милна**

$$x_{k+1} = x_{k-1} + \frac{h}{3}(f(t_{k-1}, x_{k-1}) + 4f(t_k, x_k) + f(t_{k+1}, \tilde{x}_{k+1}))$$

В методе Милна для вычисления значения $x(t_{k+1})$ необходимо последовательно применять формулы для вычисления прогноза $\tilde{x}_{k+1}$ * * и

корректора x_{k+1} , а первые четыре точки можно получить методом Рунге-Кутта.

Существует **модифицированный метод Милна**. В нём сначала вычисляется первое приближение – **прогноз Милна** $\tilde{x}_{k+1}$, затем вычисляется *управляющий параметр* m_{k+1}

$$m_{k+1} = \tilde{x}_{k+1} + \frac{28}{29}(x_k - \tilde{x}_k),$$

после чего вычисляется значение второго приближения – **корректор Милна** по формуле

$$x_{k+1} = x_{k-1} + \frac{h}{3}(f(t_{k-1}, x_{k-1}) + 4f(t_k, x_k) + f(t_{k+1}, m_{k+1}))$$

В модифицированном методе Милна первые четыре точки также можно получить методом **Рунге-Кутта**.

Рассмотрим примеры решения численного обыкновенных дифференциальных уравнений (задач Коши) с помощью методов **Эйлера**, **Рунге-Кутта**, **Милна** и **Адамса**, используя язык программирования *julia*.

Задача 1.

$$y' = y \cdot \frac{x+2}{x+1}, y(0) = 1$$

Точное решение задачи имеет вид $y = (x+1) \cdot e^x$. Ниже приведена программа на языке программирования *Julia* решения этой задачи методами **Эйлера**, **Рунге-Кутта**, **Милна** и **Адамса**.

```
In [1]: using Printf, Plots
# Правая часть дифференциального уравнения
f(x,y)=y*(x+2)/(x+1)
# Начальное условие
x0=Float64(0)
y0=Float64(1)
# Количество участков на интервале интегрирования
n=Int64(10)
# Правая граница интервала интегрирования
xn=Float64(4)
# Шаг интегрирования
h=(xn-x0)/n
# Массив абсцисс
x=zeros(n+1)
# Массив для хранения точного решения
yt=zeros(n+1)
# Цикл для формирования точного решения
for i=1:n+1
    x[i]=x0+(i-1)*h
```

```

        yt[i]=(x[i]+1)*exp(x[i])
    end
    # Реализация метода Эйлера
    # Массив для хранения решения методом Эйлера
    ye=zeros(n+1)
    ye[1]=y0
    # Цикл для формирования решения методом Эйлера
    for i=2:n+1
        ye[i]=ye[i-1]+h*f(x[i-1],ye[i-1])
    end
    # Реализация метода Рунге-Кутта
    # Массив для хранения решения методом Рунге-Кутта
    yrk=zeros(n+1)
    yrk[1]=y0
    # Цикл для формирования решения методом Рунге-Кутта
    for i=2:n+1
        K1=f(x[i-1],yrk[i-1]);
        K2=f(x[i-1]+h/2,yrk[i-1]+h/2*K1);
        K3=f(x[i-1]+h/2,yrk[i-1]+h/2*K2);
        K4=f(x[i-1]+h,yrk[i-1]+h*K3);
        delt=h/6*(K1+2*K2+2*K3+K4);
        yrk[i]=yrk[i-1]+delt;
    end
    # Реализация метода Адамса
    # Массив для хранения решения методом Адамса
    ya=zeros(n+1)
    # Формирование первых четырёх точек методом Рунге-Кутта
    for i=1:4
        ya[i]=yrk[i]
    end
    # Цикл для формирования решения методом Адамса
    for i=5:n+1
        yp=ya[i-1]+h/24*(-9*f(x[i-4],ya[i-4])+37*f(x[i-3],ya[i-3])-59*f(x[i-2],ya[i-2])+25*f(x[i-1],ya[i-1]));
        ya[i]=ya[i-1]+h/24*(f(x[i-3],ya[i-3])-5*f(x[i-2],ya[i-2])+19*f(x[i-1],ya[i-1])-9*f(x[i-4],ya[i-4]));
    end
    # Реализация метода Милна
    # Массив для хранения решения методом Милна
    ym=zeros(n+1)
    # Формирование первых четырёх точек методом Рунге-Кутта
    for i=1:4
        ym[i]=yrk[i]
    end
    # Цикл для формирования решения методом Милна
    for i=5:n+1
        yp=ym[i-4]+4*h/3*(2*f(x[i-3],ym[i-3])-f(x[i-2],ym[i-2])+2*f(x[i-1],ym[i-1])-f(x[i-4],ym[i-4]));
        ym[i]=ym[i-2]+h/3*(f(x[i-2],ym[i-2])+4*f(x[i-1],ym[i-1])+f(x[i],yp));
    end
    # Цикл для вывода результатов
    for i=1:n+1
        @printf("x=%8.2f, ye=%8.4f, yrk=%8.4f, ya=%8.4f, ym=%8.4f, yt=%8.4f\n",
    end

```

```

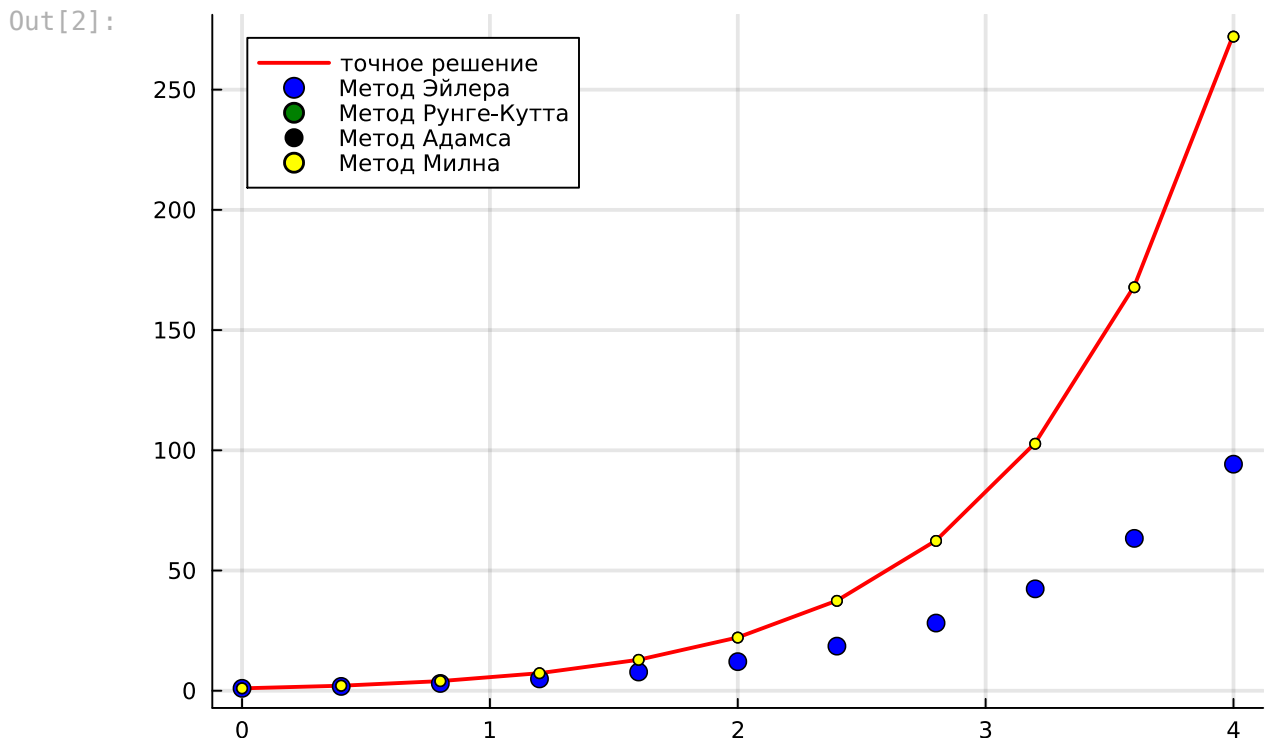
x= 0.00, ye= 1.0000, yrk= 1.0000, ya= 1.0000, ym= 1.0000, yt= 1.0000
x= 0.40, ye= 1.8000, yrk= 2.0866, ya= 2.0866, ym= 2.0866, yt= 2.0886
x= 0.80, ye= 3.0343, yrk= 4.0000, ya= 4.0000, ym= 4.0000, yt= 4.0060
x= 1.20, ye= 4.9223, yrk= 7.2905, ya= 7.2905, ym= 7.2905, yt= 7.3043
x= 1.60, ye= 7.7862, yrk= 12.8498, ya= 12.8478, ym= 12.8499, yt= 12.8779
x= 2.00, ye= 12.0985, yrk= 22.1132, ya= 22.1066, ym= 22.1129, yt= 22.1672
x= 2.40, ye= 18.5510, yrk= 37.3796, ya= 37.3635, ym= 37.3781, yt= 37.4788
x= 2.80, ye= 28.1539, yrk= 62.3124, ya= 62.2784, ym= 62.3081, yt= 62.4897
x= 3.20, ye= 42.3790, yrk=102.7267, ya=102.6597, ym=102.7166, yt=103.0366
x= 3.60, ye= 63.3668, yrk=167.8190, ya=167.6932, ym=167.7974, yt=168.3519
x= 4.00, ye= 94.2236, yrk=272.0866, ya=271.8580, ym=272.0434, yt=272.9908

```

```

In [2]: # Графическое решение задачи
Plots.plot(x, yt, color = :red, linewidth = 2, label = "точное решение", grid
Plots.scatter!(x, ye, color = :blue, markersize = 5, label = "Метод Эйлера")
Plots.scatter!(x, yrk, color = :green, markersize = 3, label = "Метод Рунге-
Plots.scatter!(x, ya, color = :black, markersize = 3, label = "Метод Адамса")
Plots.scatter!(x, ym, color = :yellow, markersize = 3, label = "Метод Милна")

```



Задача 2.

$$y' = y, y(0) = 1.$$

Точное решение этого уравнение имеет вид $y = e^x$. Решая уравнение численно методами **Эйлера**, **Рунге-Кутта**, **Милна** и **Адамса** в точке $x=1$ должны получить число $e = 2.7182818....$ Решение приведено ниже.

```

In [3]: # Правая часть дифференциального уравнения
f(x,y)=y
# Начальное условие
x0=Float64(0)

```

```

y0=Float64(1)
# Количество участков на интервале интегрирования
n=Int64(10)
# Правая граница интервала интегрирования
xn=Float64(1)
# Шаг интегрирования
h=(xn-x0)/n
# Массив абсцисс
x=zeros(n+1)
# Массив для хранения точного решения
yt=zeros(n+1)
# Цикл для формирования точного решения
for i=1:n+1
    x[i]=x0+(i-1)*h
    yt[i]=exp(x[i])
end
# Реализация метода Эйлера
# Массив для хранения решения методом Эйлера
ye=zeros(n+1)
ye[1]=y0
# Цикл для формирования решения методом Эйлера
for i=2:n+1
    ye[i]=ye[i-1]+h*f(x[i-1],ye[i-1])
end
# Реализация метода Рунге-Кутты
# Массив для хранения решения методом Рунге-Кутты
yrk=zeros(n+1)
yrk[1]=y0
# Цикл для формирования решения методом Рунге-Кутты
for i=2:n+1
    K1=f(x[i-1],yrk[i-1]);
    K2=f(x[i-1]+h/2,yrk[i-1]+h/2*K1);
    K3=f(x[i-1]+h/2,yrk[i-1]+h/2*K2);
    K4=f(x[i-1]+h,yrk[i-1]+h*K3);
    delt=h/6*(K1+2*K2+2*K3+K4);
    yrk[i]=yrk[i-1]+delt;
end
# Реализация метода Адамса
# Массив для хранения решения методом Адамса
ya=zeros(n+1)
# Формирование первых четырёх точек методом Рунге-Кутты
for i=1:4
    ya[i]=yrk[i]
end
# Цикл для формирования решения методом Адамса
for i=5:n+1
    yp=ya[i-1]+h/24*(-9*f(x[i-4],ya[i-4])+37*f(x[i-3],ya[i-3])-59*f(x[i-2],y
    ya[i]=ya[i-1]+h/24*(f(x[i-3],ya[i-3])-5*f(x[i-2],ya[i-2])+19*f(x[i-1],ya
end
# Реализация метода Милна
# Массив для хранения решения методом Милна
ym=zeros(n+1)
# Формирование первых четырёх точек методом Рунге-Кутты
for i=1:4
    ym[i]=yrk[i]
end

```

```

# Цикл для формирования решения методом Милна
for i=5:n+1
    yp=ym[i-4]+4*h/3*(2*f(x[i-3],ym[i-3])-f(x[i-2],ym[i-2])+2*f(x[i-1],ym[i-1])-f(x[i],yp));
    ym[i]=ym[i-2]+h/3*(f(x[i-2],ym[i-2])+4*f(x[i-1],ym[i-1])+f(x[i],yp));
end
# Вывод числа e, которое получено путем численного решения оду y'=y разными
@printf("x=%8.2f, ye=%8.6f, yrk=%8.6f, ya=%8.6f, ym=%8.6f, yt=%8.6f\n",

```

x= 1.00, ye=2.593742, yrk=2.718280, ya=2.718284, ym=2.718282, yt=2.718282

Задача 3.

$$y' = \frac{4}{1+x^2}, y(0) = 0.$$

Точное решение этого уравнение имеет вид $y = \arctg x$. Решая уравнение численно методами **Эйлера**, **Рунге-Кутта**, **Милна** и **Адамса** в точке $x=1$ должны получить число $\pi = 3.14159\dots$ Решение приведено ниже.

```

In [4]: # Правая часть дифференциального уравнения
f(x,y)=4/(1+x*x)
# Начальное условие
x0=Float64(0)
y0=Float64(0)
# Количество участков на интервале интегрирования
n=Int64(10)
# Правая граница интервала интегрирования
xn=Float64(1)
# Шаг интегрирования
h=(xn-x0)/n
# Массив абсцисс
x=zeros(n+1)
# Массив для хранения точного решения
yt=zeros(n+1)
# Цикл для формирования точного решения
for i=1:n+1
    x[i]=x0+(i-1)*h
    yt[i]=4*atan(x[i])
end
# Реализация метода Эйлера
# Массив для хранения решения методом Эйлера
ye=zeros(n+1)
ye[1]=y0
# Цикл для формирования решения методом Эйлера
for i=2:n+1
    ye[i]=ye[i-1]+h*f(x[i-1],ye[i-1])
end
# Реализация метода Рунге-Кутта
# Массив для хранения решения методом Рунге-Кутта
yrk=zeros(n+1)
yrk[1]=y0
# Цикл для формирования решения методом Рунге-Кутта
for i=2:n+1
    K1=f(x[i-1],yrk[i-1]);
    K2=f(x[i-1]+h/2,yrk[i-1]+h/2*K1);

```

```

    K3=f(x[i-1]+h/2,yrk[i-1]+h/2*K2);
    K4=f(x[i-1]+h,yrk[i-1]+h*K3);
    delt=h/6*(K1+2*K2+2*K3+K4);
    yrk[i]=yrk[i-1]+delt;
end
# Реализация метода Адамса
# Массив для хранения решения методом Адамса
ya=zeros(n+1)
# Формирование первых четырёх точек методом Рунге-Кутта
for i=1:4
    ya[i]=yrk[i]
end
# Цикл для формирования решения методом Адамса
for i=5:n+1
    yp=ya[i-1]+h/24*(-9*f(x[i-4],ya[i-4])+37*f(x[i-3],ya[i-3])-59*f(x[i-2],y
    ya[i]=ya[i-1]+h/24*(f(x[i-3],ya[i-3])-5*f(x[i-2],ya[i-2])+19*f(x[i-1],ya
end
# Реализация метода Милна
# Массив для хранения решения методом Милна
ym=zeros(n+1)
# Формирование первых четырёх точек методом Рунге-Кутта
for i=1:4
    ym[i]=yrk[i]
end
# Цикл для формирования решения методом Милна
for i=5:n+1
    yp=ym[i-4]+4*h/3*(2*f(x[i-3],ym[i-3])-f(x[i-2],ym[i-2])+2*f(x[i-1],ym[i-
    ym[i]=ym[i-2]+h/3*(f(x[i-2],ym[i-2])+4*f(x[i-1],ym[i-1])+f(x[i],yp));
end
# Вывод числа pi, которое получено путем численного решения оду y'=y разными
@printf("x=%8.2f, ye=%8.6f, yrk=%8.6f, ya=%8.6f, ym=%8.6f, yt=%8.6f\n",

```

x= 1.00, ye=3.239926, yrk=3.141593, ya=3.141553, ym=3.141584, yt=3.141593

Задача 4.

$$y' = \frac{1}{x}, y(1) = 0.$$

Точное решение этого уравнение имеет вид $y = \ln x$. Решая уравнение численно методами **Эйлера**, **Рунге-Кутта**, **Милна** и **Адамса** в точке $x=1$ должны получить число $\ln 2 = 0.693....$ Решение приведено ниже.

In [5]:

```

# Правая часть дифференциального уравнения
f(x,y)=1/x
# Начальное условие
x0=Float64(1)
y0=Float64(0)
# Количество участков на интервале интегрирования
n=Int64(10)
# Правая граница интервала интегрирования
xn=Float64(2)
# Шаг интегрирования
h=(xn-x0)/n
# Массив абсцисс

```

```

x=zeros(n+1)
# Массив для хранения точного решения
yt=zeros(n+1)
# Цикл для формирования точного решения
for i=1:n+1
    x[i]=x0+(i-1)*h
    yt[i]=log(x[i])
end
# Реализация метода Эйлера
# Массив для хранения решения методом Эйлера
ye=zeros(n+1)
ye[1]=y0
# Цикл для формирования решения методом Эйлера
for i=2:n+1
    ye[i]=ye[i-1]+h*f(x[i-1],ye[i-1])
end
# Реализация метода Рунге-Кутты
# Массив для хранения решения методом Рунге-Кутты
yrk=zeros(n+1)
yrk[1]=y0
# Цикл для формирования решения методом Рунге-Кутты
for i=2:n+1
    K1=f(x[i-1],yrk[i-1]);
    K2=f(x[i-1]+h/2,yrk[i-1]+h/2*K1);
    K3=f(x[i-1]+h/2,yrk[i-1]+h/2*K2);
    K4=f(x[i-1]+h,yrk[i-1]+h*K3);
    delt=h/6*(K1+2*K2+2*K3+K4);
    yrk[i]=yrk[i-1]+delt;
end
# Реализация метода Адамса
# Массив для хранения решения методом Адамса
ya=zeros(n+1)
# Формирование первых четырёх точек методом Рунге-Кутты
for i=1:4
    ya[i]=yrk[i]
end
# Цикл для формирования решения методом Адамса
for i=5:n+1
    yp=ya[i-1]+h/24*(-9*f(x[i-4],ya[i-4])+37*f(x[i-3],ya[i-3])-59*f(x[i-2],ya[i-2])+
    ya[i]=ya[i-1]+h/24*(f(x[i-3],ya[i-3])-5*f(x[i-2],ya[i-2])+19*f(x[i-1],ya[i-1])-
end
# Реализация метода Милна
# Массив для хранения решения методом Милна
ym=zeros(n+1)
# Формирование первых четырёх точек методом Рунге-Кутты
for i=1:4
    ym[i]=yrk[i]
end
# Цикл для формирования решения методом Милна
for i=5:n+1
    yp=ym[i-4]+4*h/3*(2*f(x[i-3],ym[i-3])-f(x[i-2],ym[i-2])+2*f(x[i-1],ym[i-1])-
    ym[i]=ym[i-2]+h/3*(f(x[i-2],ym[i-2])+4*f(x[i-1],ym[i-1])+f(x[i],yp));
end
# Вывод ln2, которое получено путем численного решения оду y'=y разными мето
    @printf("x=%8.2f, ye=%8.6f, yrk=%8.6f, ya=%8.6f, ym=%8.6f, yt=%8.6f\n",

```

x= 2.00, ye=0.718771, yrk=0.693147, ya=0.693153, ym=0.693149, yt=0.693147

Задача 5 (об остывании кофе). Дифференциальное уравнение, описывающее процесс остывания кофе в чашке, известно в науке, как закон теплопроводности Ньютона.

$$\frac{dT}{dt} = -r \cdot (T - T_s), T(0) = T_0 = 90$$

здесь $T(t)$ - функция определяющая температуру чашки T в момент времени t , $T_s = 23$ - температура окружающей среды, r - коэффициент остывания, для керамической чашки 250 мл - $r = 0.035 \frac{\text{Вт}}{\text{м} \cdot \text{К}}$, для керамической чашки 125 мл - $r = 0.047 \frac{\text{Вт}}{\text{м} \cdot \text{К}}$. Точное решение имеет вид:

$$T(t) = T_0 + (T_s - T_0) \cdot e^{-rt}$$

Решим уравнение численно методами **Эйлера**, **Рунге-Кутта**. Реализация на языке *Julia* приведена ниже.

```
In [6]: r=0.047
Ts=23
T0=90
# Правая часть дифференциального уравнения
f(x,y)=-r*(y-Ts)
# Начальное условие
x0=Float64(0)
y0=Float64(90)
# Количество участков на интервале интегрирования
n=Int64(10)
# Правая граница интервала интегрирования
xn=Float64(5)
# Шаг интегрирования
h=(xn-x0)/n
# Массив абсцисс
x=zeros(n+1)
for i=1:n+1
    x[i]=x0+(i-1)*h
end
# Реализация метода Эйлера
# Массив для хранения решения методом Эйлера
ye=zeros(n+1)
ye[1]=y0
# Цикл для формирования решения методом Эйлера
for i=2:n+1
    ye[i]=ye[i-1]+h*f(x[i-1],ye[i-1])
end
# Реализация метода Рунге-Кутта
# Массив для хранения решения методом Рунге-Кутта
yrk=zeros(n+1)
yrk[1]=y0
# Цикл для формирования решения методом Рунге-Кутта
for i=2:n+1
```

```

K1=f(x[i-1],yrk[i-1]);
K2=f(x[i-1]+h/2,yrk[i-1]+h/2*K1);
K3=f(x[i-1]+h/2,yrk[i-1]+h/2*K2);
K4=f(x[i-1]+h,yrk[i-1]+h*K3);
delt=h/6*(K1+2*K2+2*K3+K4);
yrk[i]=yrk[i-1]+delt;
end
# Цикл для вывода результатов
for i=1:n+1
    @printf("t=%8.2f, ye=%8.4f, yrk=%8.4f\n", x[i], ye[i], yrk[i])
end

```

```

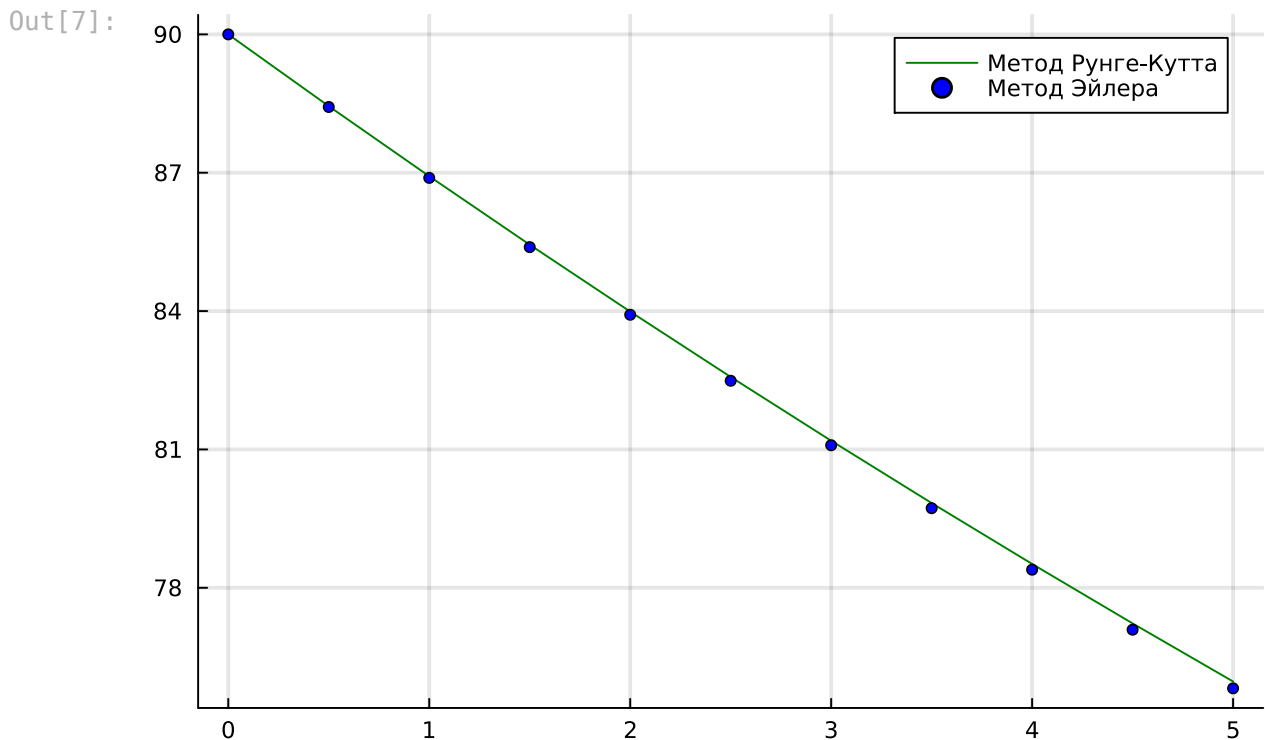
t= 0.00, ye= 90.0000, yrk= 90.0000
t= 0.50, ye= 88.4255, yrk= 88.4439
t= 1.00, ye= 86.8880, yrk= 86.9239
t= 1.50, ye= 85.3866, yrk= 85.4392
t= 2.00, ye= 83.9205, yrk= 83.9889
t= 2.50, ye= 82.4889, yrk= 82.5724
t= 3.00, ye= 81.0909, yrk= 81.1888
t= 3.50, ye= 79.7258, yrk= 79.8373
t= 4.00, ye= 78.3927, yrk= 78.5172
t= 4.50, ye= 77.0910, yrk= 77.2277
t= 5.00, ye= 75.8199, yrk= 75.9682

```

```

In [7]: # Графическое решение задачи
#plot(x, yt, color = :red, linewidth = 2, label = "точное решение",gridlines
Plots.plot(x, yrk, color = :green, markersize = 3, label = "Метод Рунге-Кутты
Plots.scatter!(x, ye, color = :blue, markersize = 3, label = "Метод Эйлера")

```



19.6 Использование пакета DifferentialEquations для решения

дифференциальных уравнений

Рассмотрим использование пакета DifferentialEquations на примере решения следующей задачи Коши

Существует описание на русском

https://engee.com/helpcenter/stable/ru/julia/DifferentialEquations/getting_started.html

Решить задачу Коши

$$\frac{dx}{dt} = 6x - 13 \cdot t^3 - 22 \cdot t^2 + 17 \cdot t - 11 + \sin(t)$$

$$x(0) = 2$$

Точное решение имеет вид

$$x(t) = \frac{119}{296} \cdot e^{6t} + \frac{1}{24} \cdot (52 \cdot t^3 + 114 \cdot t^2 - 30 \cdot t + 39) - \frac{1}{37} \cdot (6 \cdot \sin t + \cos t)$$

```
In [8]: using DifferentialEquations
using Plots
#Оперелим дифференциальное уравнение
#Правая часть дифференциального уравнения
f(x, p, t) = 6*x-13*t^3-22*t^2+17*t-11+sin(t)
#f(x, p, t) = p[1]*x+p[2]*t^3+p[3]*t^2+p[4]*t+p[4]+sin(t)
#Начальное значение
u0 = 2
# Интервал интегрирования
tspan = (0.0, 1.0)
#p=[6 -13 -22 17 -11]
# Определяем всю задачу Коши
prob = ODEProblem(f, u0, tspan)
#prob = ODEProblem(f, u0, tspan,p)
```

```
Out[8]: ODEProblem with uType Int64 and tType Float64. In-place: false
timespan: (0.0, 1.0)
u0: 2
```

```
In [9]: sol = solve(prob, Tsit5(), reltol = 1e-8, abstol = 1e-8, saveat=0.1)
#sol = solve(prob, Tsit5(), reltol = 1e-8, abstol = 1e-8, saveat=0.1, save_evs
# Параметры решателя solve
#prob - сама задача,
# Tsit5() - метод решения - в данном случае метод Цитураса 5 порядка
# могут быть другие значения
# см. описания ниже , пречисленные ниже
# reltol - относительная точность, abstol - абсолютная точность,
# save_everystep = false - сохранять или нет каждую точку найденного решения
```

```
Out[9]: retcode: Success
Interpolation: 1st order linear
t: 11-element Vector{Float64}:
 0.0
 0.1
 0.2
 0.3
 0.4
 0.5
 0.6
 0.7
 0.8
 0.9
 1.0
u: 11-element Vector{Float64}:
 2.0
 2.239126463636708
 2.8584051347890638
 4.094379683876679
 6.3672390369370735
10.4317989097424
17.65260938604915
30.50523495166459
53.48964737230455
94.79453714721119
169.3298877806762
```

Вот некоторые распространенные алгоритмы для решения ОДУ в

DifferentialEquations.jl

(https://enggee.com/helpcenter/stable/ru/julia/DifferentialEquations/getting_started.html)

- **AutoTsit5(Rosenbrock23())** подходит для решения как жестких, так и нежестких уравнений. Это неплохой алгоритм в том случае, если об уравнении ничего не известно;
- **AutoVern7(Rodas5())** подходит для эффективного решения как жестких, так и нежестких уравнений с высокой точностью;
- **Tsit5()** подходит для стандартных нежестких уравнений. Это первый алгоритм, который следует попробовать в большинстве случаев;
- **BS3()** подходит для быстрого решения нежестких уравнений с низкой точностью;
- **Vern7()** подходит для решения нежестких уравнений с высокой точностью;
- **Rodas4()** или **Rodas5()** подходят для небольших жестких уравнений с определенными в Julia типами, событиями и т. д.;
- **KenCarp4()** или **TRBDF2()** подходят для решения жестких уравнений среднего размера (100—2000 ОДУ);

- **RadauIIA5()** подходит для решения жестких уравнений с очень высокой точностью;
- **QNDF()** подходит для решения больших жестких уравнений.

Анализ решения:

- solve(x) – значение решения в точке x;
- solve[n] – обращение к значению решения номер n;
- solve.t[n] – значение абсциссы в точке номер n;

```
In [10]: sol(0.15)
```

```
Out[10]: 2.548765799212886
```

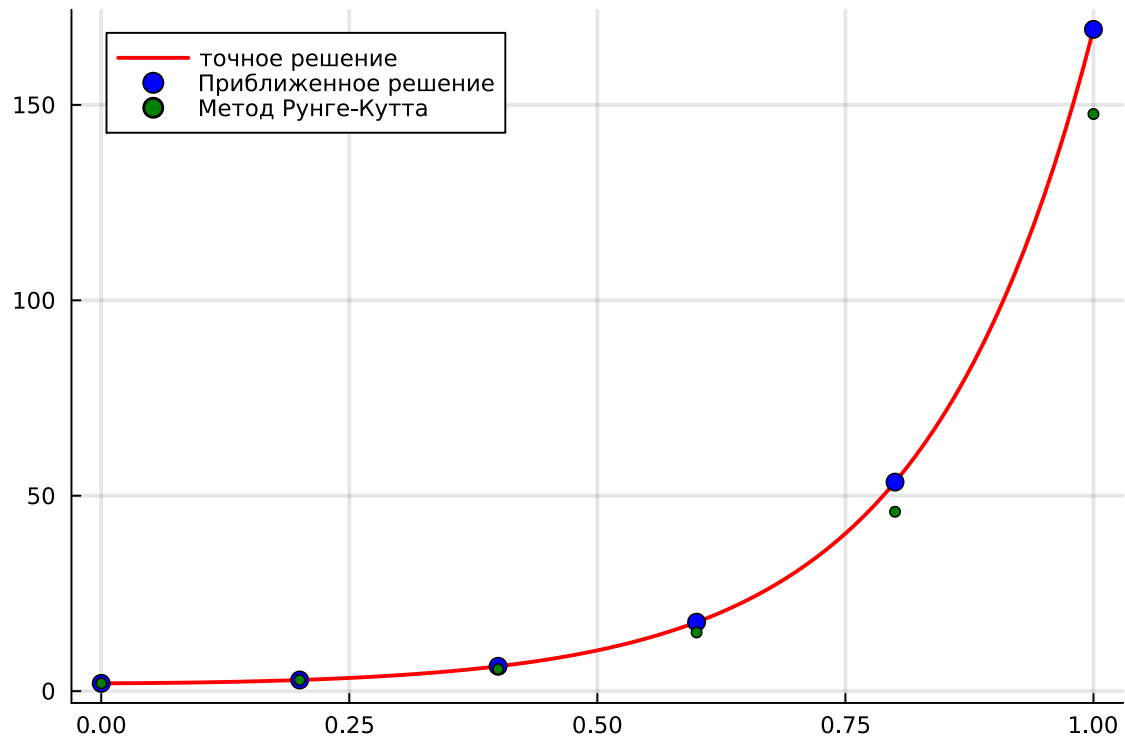
```
In [11]: g(t,x)=6*x-13*t^3-22*t^2+17*t-11+sin(t)
#точное решение
hx=0.01
n=Int64(1/hx)+1
tt=zeros(n)
yt=zeros(n)
for i=1:n
    tt[i]=(i-1)*hx
    yt[i]=119/296*exp(6*tt[i])+1/24 * (52 * tt[i]^3+114 * tt[i]^2- 30 * tt[i]
    #y[i]=sol(tt[i])
end

#приближённое решение
h2=0.2
n2=Int64(1/h2)+1
t2=zeros(n2)
y2=zeros(n2)
yRK=zeros(n2)
for i=1:n2
    t2[i]=(i-1)*h2
    y2[i]=sol(t2[i])
end
#Метод Рунге-Кутты

yRK[1]=u0;
for i=2:n2
    K1=g(tt[i-1],yRK[i-1]);
    K2=g(tt[i-1]+h2/2,yRK[i-1]+h2/2*K1);
    K3=g(tt[i-1]+h2/2,yRK[i-1]+h2/2*K2);
    K4=g(tt[i-1]+h2,yRK[i-1]+h2*K3);
    delt=h2/6*(K1+2*K2+2*K3+K4);
    yRK[i]=yRK[i-1]+delt;
end
```

```
In [12]: plot(tt, yt, color = :red, linewidth = 2, label = "точное решение",gridlines
scatter!(t2, y2, color = :blue, markersize = 5, label = "Приближенное решени
scatter!(t2, yRK, color = :green, markersize = 3, label = "Метод Рунге-Кутты
```

Out[12]:



19.7 Решение системы обыкновенных дифференциальных уравнений

Найти решение задачи Коши для следующей системы дифференциальных уравнений:

$$\frac{dx}{dt} = y \cdot z$$

$$\frac{dy}{dt} = -x \cdot z$$

$$\frac{dz}{dt} = -\frac{1}{2} \cdot z \cdot y$$

$$x(0) = 0, y(0) = 1, z(0) = 1.$$

на интервале [0; 1].

Рассмотрим решение системы обыкновенных дифференциальных уравнений методом **Эйлера**.

```
In [1]: # Правая часть системы
function fe(u,t)
    du=zeros(3)
    du[1]=u[2]*u[3];
    du[2]=-u[1]*u[3];
```

```

    du[3]=-u[1]*u[2]/2;
    return du
end
# Начальное условие
te0=Float64(0)
# Правая граница интервала интегрирования
ten=Float64(1)
ue0=[Float64(0);Float64(1);Float64(1)];
#println(ue0);
# Количество участков на интервале интегрирования
ne=Int64(25)
# Шаг интегрирования
he=(ten-te0)/ne
# Массив абсцисс
te=zeros(ne+1)
for i=1:ne+1
    te[i]=te0+(i-1)*he
end
println(te)
# Реализация метода Эйлера
# Массив для хранения решения методом Эйлера
ye=zeros(3,ne+1)
ye[:,1]=ue0
for i=2:ne+1
    ye[:,i]=ye[:,i-1]+he*fe(ye[:,i-1],te[i-1])
end
#print(ye)

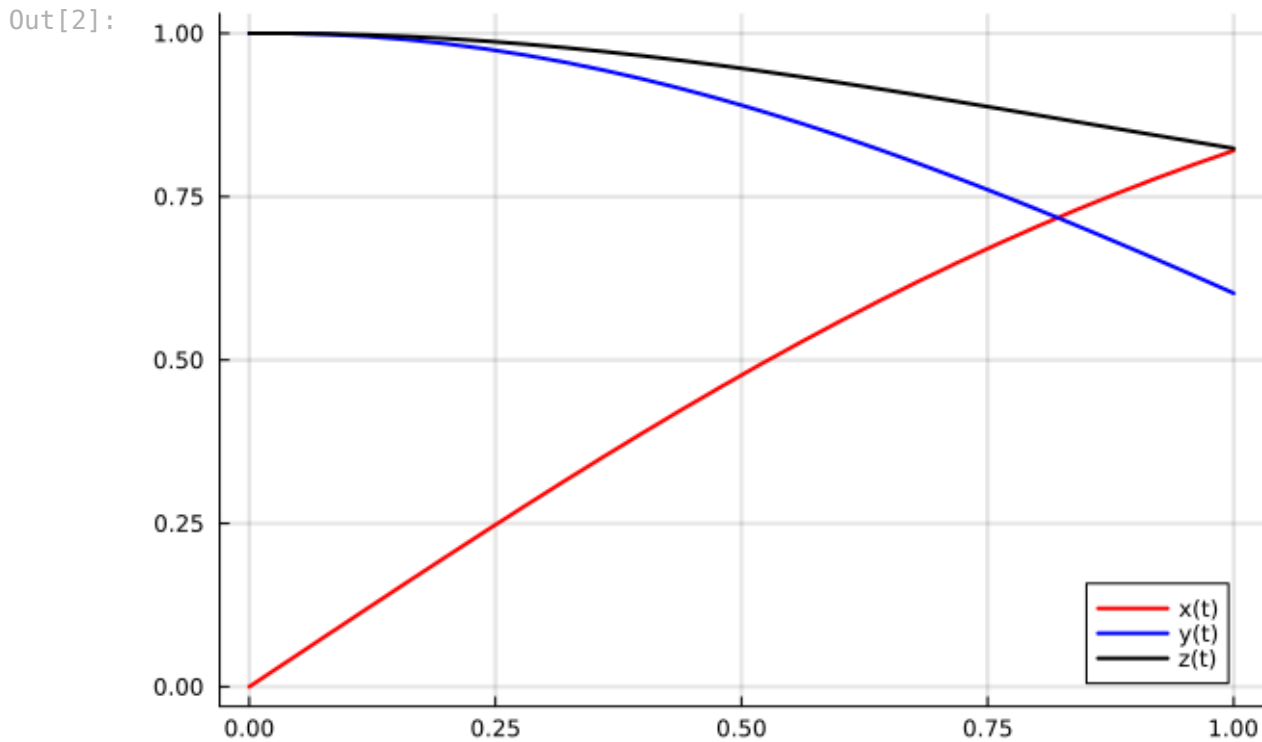
```

[0.0, 0.04, 0.08, 0.12, 0.16, 0.2, 0.24, 0.28, 0.32, 0.36, 0.4, 0.44, 0.48, 0.52, 0.56, 0.6, 0.64, 0.68, 0.72, 0.76, 0.8, 0.84, 0.88, 0.92, 0.96, 1.0]

Самостоятельное задание. Реализуйте метод **Рунге-Кутта** для рассматриваемой системы.

График решения, найденного методом **Эйлера**.

In [2]: `using Plots`
`Plots.plot(te, ye[1,:], color = :red, linewidth = 2, label = "x(t)",gridline`
`Plots.plot!(te, ye[2,:], color = :blue, linewidth = 2, label = "y(t)",gridli`
`Plots.plot!(te, ye[3,:], color = :black, linewidth = 2, label = "z(t)",gridl`



А теперь найдём решение используя пакет DifferentialEquations.

```
In [4]: function f(du, u, t)
        du[1]=u[2]*u[3];
        du[2]=-u[1]*u[3];
        du[3]=-u[1]*u[2]/2;
    end
```

Out[4]: f (generic function with 1 method)

```
In [5]: using DifferentialEquations
        function syst1(du, u, p, t)
            du[1]=u[2]*u[3];
            du[2]=-u[1]*u[3];
            du[3]=-u[1]*u[2]/2;
        end
        u0 = [0; 1; 1]
        tspan = (0, 1)
        prob = ODEProblem(syst1, u0, tspan)
        sol = solve(prob, saveat=0.1)
        #sol = solve(prob, AutoTsit5(Rosenbrock23()), reltol = 1e-8, abstol = 1e-8)
        #sol = solve(prob, Rodas5(), reltol = 1e-8, abstol = 1e-8)
```

ArgumentError: Package DifferentialEquations not found in current path.
- Run `import Pkg; Pkg.add("DifferentialEquations")` to install the DifferentialEquations package.

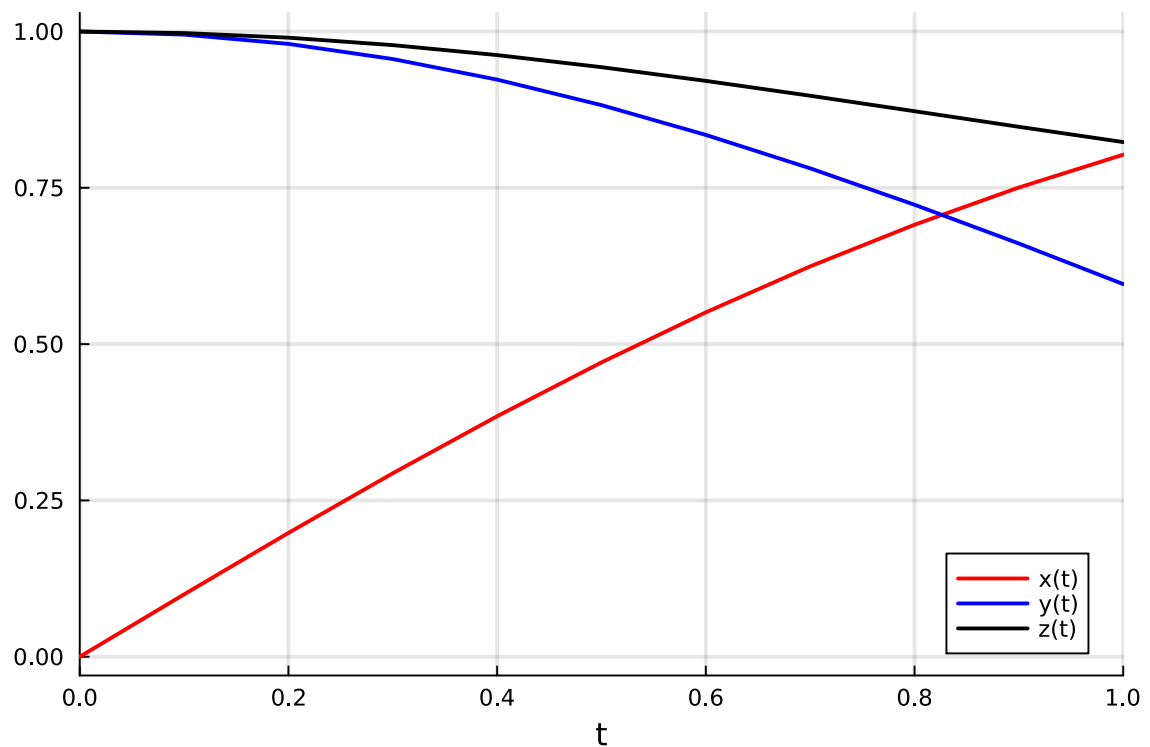
Stacktrace:

```
[1] macro expansion
  @ ./loading.jl:2375 [inlined]
[2] macro expansion
  @ ./lock.jl:376 [inlined]
[3] __require(into::Module, mod::Symbol)
  @ Base ./loading.jl:2358
[4] require(into::Module, mod::Symbol)
  @ Base ./loading.jl:2334
[5] eval(m::Module, e::Any)
  @ Core ./boot.jl:489
```

In [17]: *#Построение графика решения*

```
using Plots
Plots.plot(sol, idxs=(0,1), color = :red, linewidth = 2, label = "x(t)", grid=:true)
Plots.plot!(sol, idxs=(0,2), color = :blue, linewidth = 2, label = "y(t)", grid=:true)
Plots.plot!(sol, idxs=(0,3), color = :black, linewidth = 2, label = "z(t)", grid=:true)
```

Out[17]:



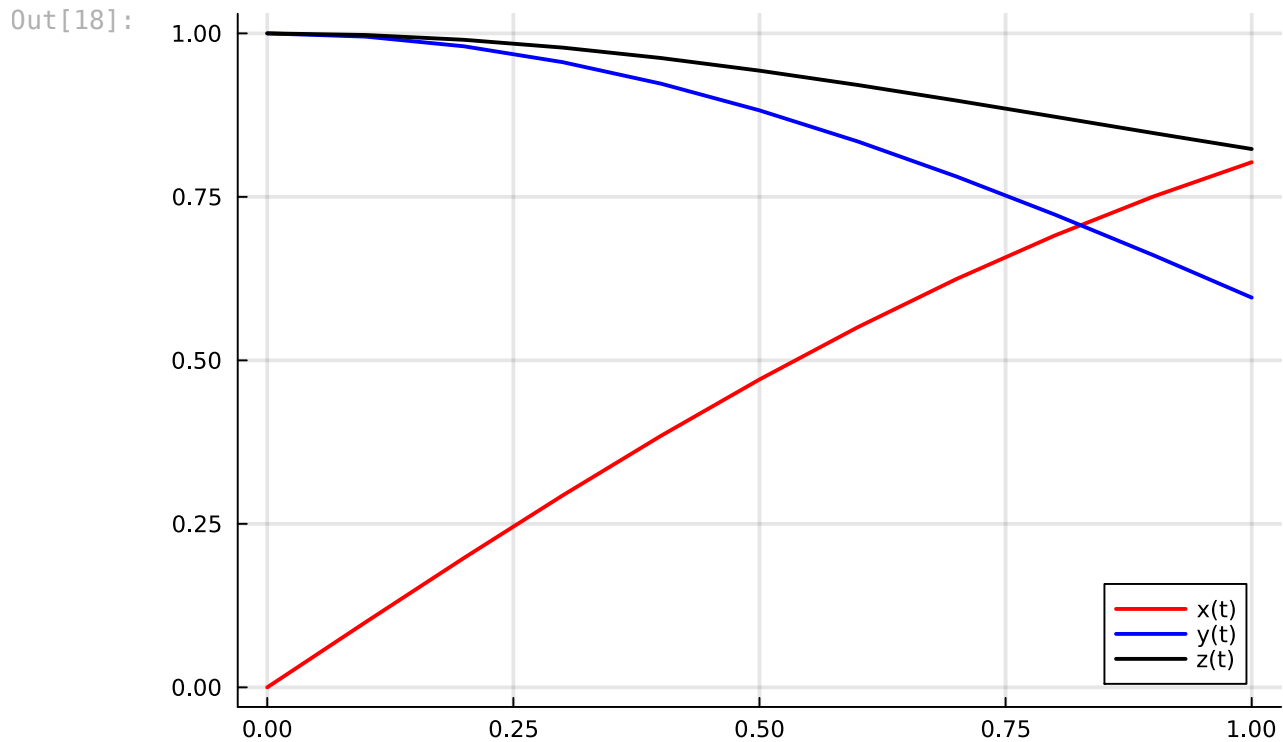
In [18]: *#Другая версия построения графика решения*

```
ttt=zeros(11)
x=zeros(11)
y=zeros(11)
z=zeros(11)
for i=1:11
    ttt[i]=sol.t[i]
    x[i]=sol.u[i][1]
    y[i]=sol.u[i][2]
```

```

    z[i]=sol.u[i][3]
end
Plots.plot(ttt, x, color = :red, linewidth = 2, label = "x(t)",gridlinewidth=1)
Plots.plot!(ttt, y, color = :blue, linewidth = 2, label = "y(t)",gridlinewidth=1)
Plots.plot!(ttt, z, color = :black, linewidth = 2, label = "z(t)",gridlinewidth=1)

```



Наложим точки, найденные с помощью метода **Эйлера** на график решения, полученный с помощью **solve**.

```

In [19]: Plots.plot(ttt, x, color = :red, linewidth = 2, label = "x(t)",gridlinewidth=1)
Plots.scatter!(te, ye[1,:], color = :red, markersize = 4, label = "Метод Эйлера")
Plots.plot!(ttt, y, color = :blue, linewidth = 2, label = "y(t)",gridlinewidth=1)
Plots.scatter!(te, ye[2,:], color = :blue, markersize = 4, label = "Метод Эйлера")
Plots.plot!(ttt, z, color = :black, linewidth = 2, label = "z(t)",gridlinewidth=1)
Plots.scatter!(te, ye[3,:], color = :black, markersize = 4, label = "Метод Эйлера")

```

Out[19]:

